



LeetCode 19: Remove Nth Node From End of List

1. Problem Title & Link

Title: LeetCode 19: Remove Nth Node From End of List

Link: <https://leetcode.com/problems/remove-nth-node-from-end-of-list/>

2. Problem Statement (Short Summary)

Given the head of a linked list and an integer n , remove the **nth node from the end** of the list and return the head.

The deletion must be done in one traversal if possible.

3. Examples (Input → Output)

Example 1

Input:

head = [1,2,3,4,5]

$n = 2$

Output:

[1,2,3,5]

Explanation:

2nd node from end = 4

Delete 4.

Example 2

Input:

head = [1]

$n = 1$

Output:

[]

Example 3

Input:

head = [1,2]

$n = 1$

Output:

[1]

4. Constraints

- Number of nodes: $1 \leq \text{sz} \leq 30$



- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq \text{sz}$

5. Core Concept (Pattern / Topic)

Fast & Slow Pointer

Related Topics:

- Linked List
- Two Pointers
- One Pass Traversal

6. Thought Process (Step-by-Step Explanation)

Brute Force

1. Find length of linked list.
2. Compute:

length - n

3. Traverse again and delete node.

Complexity:

$O(2N)$

Optimized Approach

Use:

Fast Pointer

Slow Pointer

Move fast pointer:

n + 1 steps

ahead.

Then move both pointers together.

When fast reaches:

null

slow will be exactly before the node to delete.

Why Dummy Node?

Consider:

[1]

Deleting head.

Without dummy node special handling is required.

Dummy simplifies all cases.



7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
1 → 2 → 3 → 4 → 5
n = 2
Move fast:
D → 1 → 2 → 3
      ↑
      fast

Slow:
D
↑
slow
Move together:
D → 1 → 2 → 3 → 4 → 5
      ↑
      slow

Delete:
slow.next = slow.next.next
```

8. Pseudocode

```
Create dummy node

fast = dummy
slow = dummy

Move fast n+1 steps

while fast != null:
    fast = fast.next
    slow = slow.next

Delete slow.next

return dummy.next
```

9. Code Implementation

 Python

```
class Solution:
    def removeNthFromEnd(self, head, n):

        dummy = ListNode(0)
        dummy.next = head

        fast = dummy
```



```
slow = dummy

for _ in range(n + 1):
    fast = fast.next

while fast:

    fast = fast.next
    slow = slow.next

slow.next = slow.next.next

return dummy.next
```

✓ Java

```
class Solution {

    public ListNode removeNthFromEnd(
        ListNode head,
        int n) {

        ListNode dummy =
            new ListNode(0);

        dummy.next = head;

        ListNode fast = dummy;
        ListNode slow = dummy;

        for(int i=0;i<=n;i++) {
            fast = fast.next;
        }

        while(fast != null) {

            fast = fast.next;
            slow = slow.next;
        }

        slow.next = slow.next.next;

        return dummy.next;
    }
}
```

10. Time & Space Complexity

Time Complexity



$O(N)$

Single traversal.

Space Complexity

$O(1)$

11. Common Mistakes / Edge Cases

Mistake 1

Not using dummy node.

Fails when deleting head.

Mistake 2

Moving fast only n steps.

Need:

$n + 1$

to position slow correctly.

Mistake 3

Null pointer issues.

12. Detailed Dry Run

Input:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

$n = 2$

After moving fast:

fast at 3

slow at dummy

Move together:

Fast	Slow
4	1
5	2
null	3

Delete:

`slow.next = slow.next.next`

Result:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 5$



13. Common Use Cases

Streaming Data

Remove kth latest event.

Linked List Editing

Delete node relative to end.

Queue Systems

Remove nth request from tail.

14. Common Traps

- Forgetting dummy node
- Wrong fast pointer distance
- Deleting wrong node

15. Builds To (Related Problems)

1. LC 876 – Middle of Linked List
2. LC 141 – Linked List Cycle
3. LC 19 – Remove Nth Node From End
4. LC 143 – Reorder List
5. LC 234 – Palindrome Linked List

16. Alternate Approaches

Approach	Time	Space	Notes
Length Count	$O(N)$	$O(1)$	Two Pass
Fast Slow Pointer	$O(N)$	$O(1)$	Optimal

17. Why This Solution Works

Maintaining a gap of $n+1$ nodes between fast and slow ensures that when fast reaches the end, slow is exactly before the node that must be deleted.

18. Variations / Follow-Up Questions

Follow-Up 1

Delete kth node from start.

Simple traversal.

Follow-Up 2



Delete middle node.

Use Fast-Slow Pointer.

Follow-Up 3

Delete multiple nodes from end.

Can use stack or repeated traversal.